Exercise: Comparing Optimizers in an Artificial Neural Network (ANN)

In this exercise, we will build and train an Artificial Neural Network (ANN) using TensorFlow/Keras and compare how different optimization algorithms affect the learning process of the model.

Training a neural network involves adjusting the weights of the network to minimize the loss function. The method used to update these weights is called an optimizer. Different optimizers use different strategies to update the weights, which can impact the speed of convergence, stability of training, and final accuracy of the model.

The objective of this exercise is to understand how different optimizers behave during training and how they influence the performance of a neural network.

Learning Objectives

After completing this exercise, you will be able to:

- 'Understand the basic workflow of training an Artificial Neural Network'
- 'Explain the role of optimizers in neural network training'
- 'Understand why feature scaling is important for neural networks'
- 'Compare the performance of different optimizers such as SGD, RMSprop, and Adam'
- 'Interpret training loss and accuracy graphs'
- 'Understand how hyperparameters such as learning rate, batch size, and number of epochs affect model performance'

1.Import necessary Libraries

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense,Input
from tensorflow.keras.optimizers import Adam, SGD,Adagrad,RMSprop
```

Importing all the libraries needed for the experiment. We need deep learning libraries to build the ANN, data handling libraries to organize results, plotting libraries to visualize learning, and scikit-learn utilities for loading and preprocessing the dataset.

Why these imports are used

- `tensorflow as tf` : main deep learning framework used to train the ANN.
- `numpy as np` : helps with numerical operations on arrays.
- `matplotlib.pyplot as plt` : used to plot loss and accuracy graphs.
- `pandas as pd` : used to create a comparison table of optimizer results.
- `load_breast_cancer` : loads the breast cancer dataset.
- `train_test_split` : splits the dataset into training and testing parts.
- `StandardScaler` : standardizes input features so learning becomes faster and more stable.
- `Sequential` : lets us stack neural network layers one after another.
- `Dense` : creates fully connected neural network layers.
- `Input` : defines the shape of the input layer.
- `SGD`, `Adagrad`, `RMSprop`, `Adam` : different optimizers used for comparison.

Questions with answers

Q1. Why do we import libraries instead of writing everything from scratch?
Because these libraries already provide efficient, tested implementations of common machine learning tasks.

Q2. Why do we use TensorFlow/Keras here?
Because it makes ANN model building, training, and evaluation easier.

Q3. Why is Matplotlib needed?
To visualize how loss and accuracy change during training.

Q4. Why do we use Pandas?
To present optimizer results in a neat tabular format.

2.Data Preprocessing

```
# Load Dataset
data = load_breast_cancer()
X = data.data
```

```
y = data.target
```

Loading the breast cancer dataset and separates it into input features `X` and target labels `y`.

Why each line is used

- `data = load_breast_cancer()` : loads a ready-made binary classification dataset.
- `X = data.data` : stores the feature values used for prediction.
- `y = data.target` : stores the target labels, where the model learns to classify the samples.
- `data` : displays dataset information such as features and target names.

Questions with answers

Q1. What type of problem is this dataset used for?
It is a binary classification problem.

Q2. What does `X` represent?
`X` contains the input features used by the ANN.

Q3. What does `y` represent?
`y` contains the class labels the model must predict.

Q4. Why should we understand the dataset before training?
Because knowing the data type, feature meaning, and target helps us choose the correct model and loss function.

```
#  Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Splitting the dataset into training data and testing data. The model learns from the training data and is evaluated on the testing data.

Why each parameter is used

- `X, y` : the input features and labels to be split.
- `test_size=0.2` : reserves 20% of the dataset for testing.
- `random_state=42` : ensures the same split every time the notebook runs, which improves reproducibility.

Questions with answers

Q1. Why do we split the dataset?
To test the model on unseen data and check whether it generalizes well.

Q2. What happens if we train and test on the same data?
The model may appear to perform well but may fail on new data.

Q3. Why do we use `test_size=0.2`?
It keeps most data for learning while reserving enough data for evaluation.

Q4. Why is `random_state` important?
It makes the data split reproducible.

```
# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Scale the input features using standardization. Neural networks generally train better when input values are on a similar scale.

Why each line is used

- `scaler = StandardScaler()` : creates a scaler object that standardizes features.
- `fit_transform(X_train)` : learns the mean and standard deviation from training data and transforms it.
- `transform(X_test)` : applies the same scaling learned from training data to testing data.

Questions with answers

Q1. Why is feature scaling important for ANNs?
Because large feature values can dominate the learning process and slow down convergence.

Q2. Why do we fit the scaler only on training data?
To avoid data leakage from the test set.

Q3. What does standardization do?
It changes each feature so it has approximately mean 0 and standard deviation 1.

Q4. What can happen if we do not scale the features?
Training may become slower and less stable.

3.Model Building

3.1 Function to Create Model

```
# Function to Create Model
def create_model(optimizer):
    model = Sequential([
        Input(shape=(X_train.shape[1],)),
        Dense(16, activation='relu'),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
    return model
```

Defining a function that creates the ANN model. We use a function so that the same architecture can be rebuilt for each optimizer.

Why each part is used

- `create_model(optimizer)` : accepts the optimizer as an input so the same network can be trained with different optimizers.
- `Input(shape=(X_train.shape[1],))` : sets the number of input neurons equal to the number of features.
- `Dense(16, activation='relu')` : first hidden layer with 16 neurons and ReLU activation.
- `Dense(8, activation='relu')` : second hidden layer with 8 neurons and ReLU activation.
- `Dense(1, activation='sigmoid')` : output layer with one neuron because this is binary classification.
- `optimizer=optimizer` : uses whichever optimizer is passed to the function.
- `loss='binary_crossentropy'` : suitable loss for binary classification.
- `metrics=['accuracy']` : tracks accuracy during training.

Questions with answers

Q1. Why do we use hidden layers?
Hidden layers help the network learn complex patterns and nonlinear relationships in the data.

Q2. Why do we use ReLU in hidden layers?
ReLU is simple, fast, and helps reduce the vanishing gradient problem.

Q3. Why do we use sigmoid in the output layer?
Because sigmoid gives an output between 0 and 1, which is suitable as a probability for binary classification.

Q4. Why do we use binary cross-entropy?
Because the task has only two classes, and binary cross-entropy measures how well predicted probabilities match the true binary labels.

Q5. Why do we create a function instead of writing the model repeatedly?
It avoids repetition and makes optimizer comparison cleaner and more consistent.

3.2 Define Optimizers

```
#  Define Optimizers
optimizers = {
    "SGD": SGD(learning_rate=0.01),
    "Momentum": SGD(learning_rate=0.01, momentum=0.9),
    "AdaGrad": Adagrad(learning_rate=0.01),
    "RMSProp": RMSprop(learning_rate=0.001),
    "Adam": Adam(learning_rate=0.001)
}

histories = {}
results = []

# histories = {} → stores training history of each optimizer (used for graph

# results = [] → stores final loss and accuracy values (used for numerical c
```

Defining the optimizers that will be compared and prepares variables to store histories and final results.

Why each parameter is used

- `SGD(learning_rate=0.01)` : standard gradient descent with a fixed step size.
- `momentum=0.9` : helps SGD move faster in useful directions and reduces oscillation.
- `Adagrad(learning_rate=0.01)` : adapts the learning rate for each parameter.

- `RMSprop(learning_rate=0.001)` : adjusts learning rates using a moving average of squared gradients.
- `Adam(learning_rate=0.001)` : combines momentum and adaptive learning rates.
- `histories = {}` : stores training history for plotting.
- `results = []` : stores final test loss and accuracy values.

Questions with answers

Q1. Why do we compare multiple optimizers?
Because different optimizers can train the same ANN differently in terms of speed and final accuracy.

Q2. What is the role of learning rate?
It controls how big a step the optimizer takes during weight updates.

Q3. Why does Momentum usually improve over plain SGD?
Because it carries forward part of the previous update and helps move faster through shallow regions.

Q4. Why is Adam popular?
Because it often converges faster and works well on many problems without much tuning.

3.3 Training Loop

```
# Training Loop
for name, opt in optimizers.items():
    print(f"\nTraining with {name} optimizer...")

    # Create a new model for this optimizer
    model = create_model(opt)

    # Train the model
    history = model.fit(
        X_train, y_train,
        epochs=30,
        batch_size=32,
        validation_data=(X_test, y_test),
        verbose=0
    )

    histories[name] = history

    # Evaluate the model on test data
    loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

    # Append results for table
```

```
        results.append({"Optimizer": name, "Test Loss": loss, "Test Accuracy": a
```

```
Training with SGD optimizer...

Training with Momentum optimizer...

Training with AdaGrad optimizer...

Training with RMSProp optimizer...

Training with Adam optimizer...
```

Training the same ANN architecture using each optimizer, stores the training history, and evaluates the model on the test data.

Why each parameter is used

- `for name, opt in optimizers.items()` : loops through all optimizers.
- `tf.keras.backend.clear_session()` : clears old TensorFlow state before building a fresh model.
- `model = create_model(opt)` : creates a new ANN using the current optimizer.
- `epochs=30` : trains the model for 30 complete passes through the training data.
- `batch_size=32` : updates weights after every 32 samples.
- `validation_data=(X_test, y_test)` : checks performance on unseen data after each epoch.
- `verbose=0` : suppresses detailed training output.
- `model.evaluate(...)` : computes final test loss and accuracy.

Questions with answers

Q1. Why do we create a new model for each optimizer?
Because each optimizer must train from the same starting architecture for a fair comparison.

Q2. What is an epoch?
One complete pass of the entire training dataset through the model.

Q3. What is batch size?
The number of training samples used before one weight update happens.

Q4. Why do we use validation data during training?
To monitor how well the model performs on unseen data and detect overfitting.

3.4 Comparison of Optimizers

```
#  Display Results in a Pandas Table
df_results = pd.DataFrame(results)
# This line sorts the optimizer results by test accuracy in descending order
df_results = df_results.sort_values(by="Test Accuracy", ascending=False).res
print("\n===== Numerical Comparison of Optimizers =====\n")
print(df_results)
```

```
===== Numerical Comparison of Optimizers =====

  Optimizer  Test Loss  Test Accuracy
0   AdaGrad   0.073226       0.973684
1   RMSProp   0.069615       0.973684
2      Adam   0.064330       0.973684
3       SGD   0.126108       0.956140
4  Momentum   0.107973       0.956140
```

Converting the stored results into a table and sorts them by test accuracy so the best optimizer appears at the top.

Why each line is used

- `pd.DataFrame(results)` : converts the results list into a readable table.
- `sort_values(by="Test Accuracy", ascending=False)` : sorts from highest test accuracy to lowest.
- `reset_index(drop=True)` : resets row numbers after sorting.
- `print(df_results)` : displays the comparison table.

Questions with answers

Q1. Why do we convert results into a DataFrame?
Because tables are easier to read and compare than raw lists.

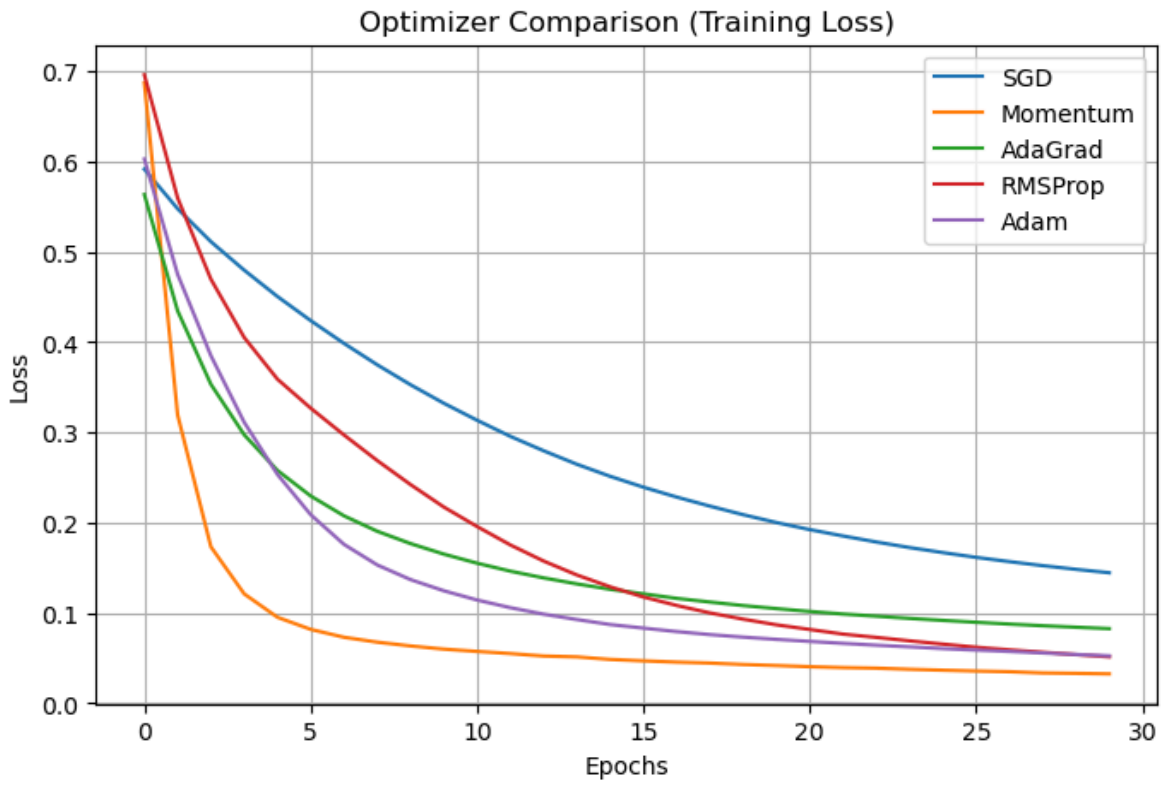
Q2. Why do we sort by test accuracy?
To quickly identify which optimizer performed best on unseen data.

Q3. Why do we use test accuracy instead of training accuracy here?
Because test accuracy better reflects how well the model generalizes.

Q4. What else can we compare besides accuracy?
We can compare loss, training speed, and stability.

3.5 Plot Training Loss

```
# Plot Training Loss
plt.figure(figsize=(8,5))
for name in histories:
    plt.plot(histories[name].history['loss'], label=name)
plt.title("Optimizer Comparison (Training Loss)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```



Plotting training loss for all optimizers across epochs so we can compare how quickly and how smoothly each optimizer learns.

Why each parameter is used

- `plt.figure(figsize=(8,5))` : creates a figure with a readable size.
- `histories[name].history['loss']` : gets loss values recorded during training.

- `label=name` : gives each optimizer a label in the legend.
- `plt.title(...)`, `plt.xlabel(...)`, `plt.ylabel(...)` : make the graph easier to understand.
- `plt.legend()` : shows which line belongs to which optimizer.
- `plt.grid(True)` : improves readability.

Questions with answers

Q1. What does loss tell us?
Loss tells us how far the model's predictions are from the true values.

Q2. What does it mean if the loss decreases?
It means the model is learning and predictions are improving.

Q3. Why do we compare optimizer loss curves?
To see which optimizer learns faster and more stably.

Q4. What does a very unstable loss curve indicate?
It may indicate poor convergence or an unsuitable learning rate.

Observation from the graph

Momentum (orange line)

-Drops very quickly in the first few epochs. -Ends with the lowest loss (~0.04). -Best performing optimizer in this plot.

RMSProp (red line) -Also converges quickly. -Final loss around ~0.05. -Second best.

Adam (purple line) -Smooth and stable convergence. -Final loss around ~0.06. -Third best.

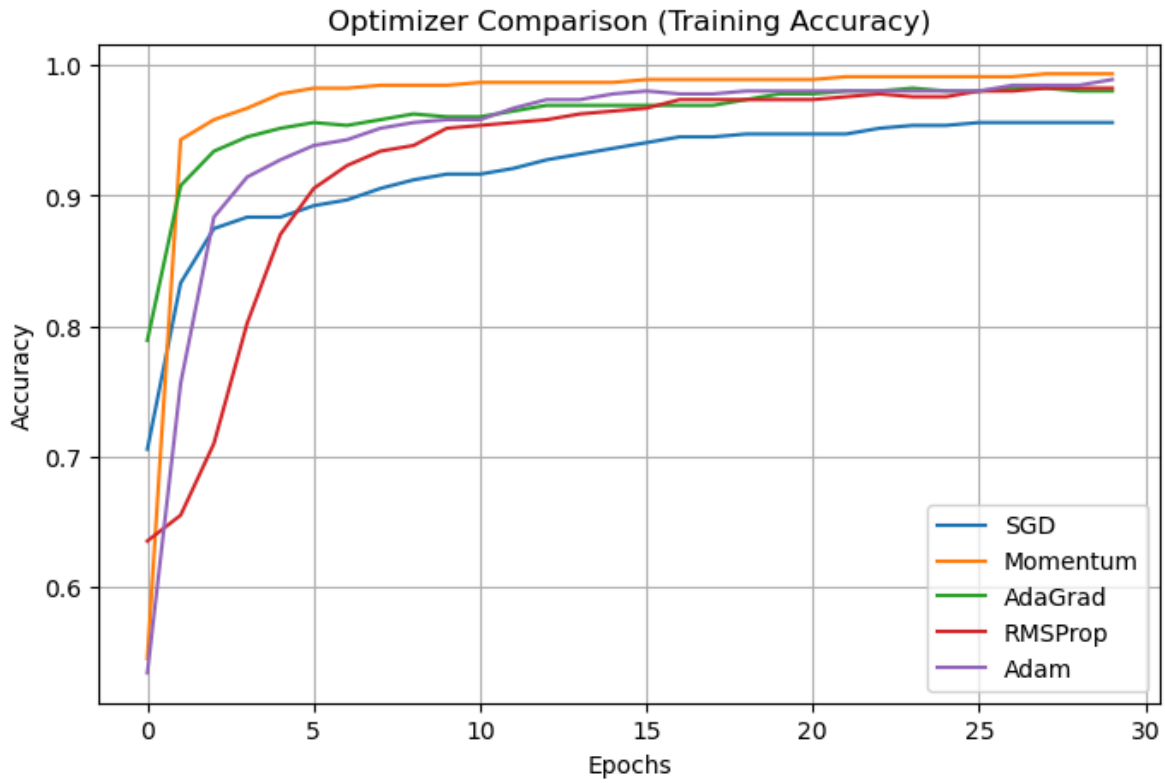
SGD (blue line) -Decreases slowly. -Final loss around ~0.15. -Slower convergence.

AdaGrad (green line) -Loss decreases initially but then slows down significantly. -Final loss around ~0.27. -Worst performance here.

3.6 Plot Training Accuracy

```
#  Plot Training Accuracy
plt.figure(figsize=(8,5))
for name in histories:
    plt.plot(histories[name].history['accuracy'], label=name)
plt.title("Optimizer Comparison (Training Accuracy)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



plots training accuracy for all optimizers across epochs, allowing us to compare how quickly each optimizer improves prediction correctness.

Why each parameter is used

- `histories[name].history['accuracy']` : extracts accuracy values from training history.
- The remaining plotting commands are used to label and clearly display the graph.

Questions with answers

Q1. What does accuracy represent?
Accuracy is the proportion of correct predictions out of all predictions.

Q2. Why is training accuracy alone not enough?
Because a model can perform well on training data but poorly on unseen data.

Q3. Why compare accuracy curves for different optimizers?
To observe which optimizer reaches better performance faster.

Q4. How can graphs help detect overfitting?

If training performance improves but validation performance stops improving, overfitting may be happening.